

Type-Theoretic SSA

Premature optimization is the root of all evil.

— Donald Knuth

The goal of this chapter is to give a formal presentation of SSA – fully specifying its *syntax*, *type theory*, and *semantics*. Rather than do this all at once, we’re instead going to “build up to” SSA by studying a sequence of increasingly complex languages, each of which embeds into the next.

In particular, we’ll begin by introducing:

1. The language of simple arithmetic expressions, λ_{arith} , which we’ll extend to:
2. The language of *effectful* expressions, λ_{expr} , which we’ll restrict to:
3. The language of *operations*, λ_{op} – a single effectful expression which depends on variables and constants. This will allow us to define
4. The language of *basic blocks*, λ_{bb} , – straight-line sequences of operations – which we’ll show *equivalent* to the language of effectful expressions with *let-bindings*. By adding conditional branches, we can then recover
5. The language of *acyclic regions*, λ_{areg} , – single-entry, single-exit *acyclic* control-flow graphs of basic blocks – which we’ll show *equivalent* to the language of effectful expressions with *case-expressions*

Finally, we’ll drop the acyclicity restriction to get to our final type-theoretic presentation of SSA:

6. The language of *regions*, λ_{reg} , – single-entry, multiple-exit control-flow graphs of basic blocks – which we’ll show *equivalent* to the language of effectful expressions with *iteration*

Our goal is to give a precise mathematical description of each of these languages – consequently, each language’s section will follow a somewhat repetitive pattern, where we specify:

- A *syntax*, which specifies how to tell whether a term is *grammatical* – like “ $x * (y + z)$ ” – or nonsense – like “ $C + +$ ”.
- A *type system*, which consists of:
 - A collection of ways we can *interpret* a given term (natural number, string, graph, tensor, neural network...) – that is, a set of *types* A, B, C
 - A description of the *contexts* in which we can interpret terms
 - A *typing relation* – which tells us in which contexts a given term can be interpreted in a given way.

Concretely: we might be able to interpret $x + x$ as an integer in the context $x : \mathbb{Z}$, or as a string in the context $x : \text{str}$ – but $x + x$ doesn’t make sense for $x : \text{POBox}$.

Normally, a *type theory* consists of a type system together with an *equational theory*: a relation which tells us when two terms are “the same” in a particular context – for example, we might have $x + y \equiv y + x$ in the context $x : \mathbb{Z}, y : \mathbb{Z}$, but not in the context $x : \text{str}, y : \text{str}$.

This can let us reason about programs, but also *optimize* them – for example, we might rewrite

$$(x + 5) + (x + 5) + (x + 5) \quad \longrightarrow \quad \text{let } y = x + 5; 3 \cdot y$$

Since our main focus will be the study of compiler optimizations, we will hence generalize equational theories to *refinement theories*: where a refinement theory is a relation which tells us that replacing a term t with another term t' is always sound, i.e., that t' *refines* t – written $t \rightarrow t'$.

We can recover an equational theory by saying that t and t' are *equivalent* – written $t \approx t'$ – whenever $t \rightarrow t'$ and $t' \rightarrow t$. Likewise, every equational theory induces a refinement theory by simply defining $(t \rightarrow t') := (t \approx t')$.

As a concrete example, let's assume that the result of $x/0$ is *implementation-defined* – and, in particular, may return a different result each time it's evaluated.

Then it's sound to rewrite

$$(x/y) + (x/y) \longrightarrow \text{let } z = x/y; z + z$$

since, whether or not $y = 0$, every valid result for the original program is also a valid result for the rewritten program.

On the other hand, for $x, y : \mathbb{Z}$, the rewrite

$$\text{let } z = x/y; 2 \cdot z \not\longrightarrow (x/y) + (x/y)$$

would be unsound, since every valid result for the left-hand side is *even* (since it can be written $2 \cdot z$ for some z) but the right-hand side can be *odd* – for example, if $y = 0$, we could have the reduction sequence

$$(x/y) + (x/y) \longrightarrow 1 + (x/y) \longrightarrow 1 + 0 \longrightarrow 1$$

Just like equations, refinements can be *context-dependent* – for example, depending on our semantics, we may have that

$$\text{let } z = x/y; 2 \cdot z \longrightarrow (x/y) + (x/y)$$

is a perfectly valid optimization for $x, y : \mathbb{R}$.

At this point, we've got a *theory* for our programming language:

- A *syntax*, which sets the domain of discourse
- A *type theory*, which tells us what rules we can use to reason about sentences in our syntax

What we now need to confirm is whether this theory is a *good* one – that is, whether it captures our intuitions about how programs should behave.

To do this, we need to give our theory a *semantics*. There are two primary approaches we might take:

- We can provide an *operational semantics*: a relation which specifies the behaviour of a program by describing how it would be executed by an *abstract machine*.

Our semantics is then *sound* if $t \rightarrow t'$, implies that valid execution of t' is a valid execution of t .

Alternatively,

- We can provide a *denotational semantics*: a mapping from well-formed terms in our syntax to mathematical objects.

Soundness in this case means that every sentence which is true according to our type theory is true when interpreted as a statement about the semantics.

For example, if we interpret programs like $x/(y+z)$ as functions from *valuations* $\text{Var} \rightarrow \mathbb{Z}$ to sets of possible results $\mathcal{P}(\mathbb{Z})$, then we might say our type theory is *sound* if, for all well-formed t, t' ,

$$t \rightarrow t' \implies \llbracket t \rrbracket \subseteq \llbracket t' \rrbracket \quad (\text{pointwise})$$

The standard approach for reasoning about SSA – e.g. in **TODO 1: VELVVM, etc** – is operational. One of the primary contributions of this thesis is to give a *compositional* denotational semantics for SSA – that is, a denotational semantics in which the semantics of a program as a whole is a function of the semantics of its parts. In particular, this will be enabled by *categorical semantics*: where types and contexts are interpreted as *objects* in a *category*.

While the resulting semantics enables compositional reasoning about SSA programs, it is also quite abstract – hence, to validate that it is in fact the “right” semantics, we’ll show that:

- It is *complete* with respect to our refinement theory: that is, $t \rightarrow t'$ if and only if $\llbracket t \rrbracket_{\mathcal{M}} \leq \llbracket t' \rrbracket_{\mathcal{M}}$ for every model $\mathcal{M}$
- Our refinement theory is *sound* with respect to a reasonable operational semantics – and hence our categorical semantics at least *abstracts* the operational semantics.

Let’s get started!

1. Basic Arithmetic

We’ll do it tomorrow

– TODO

We’ll start by giving a formal account of the simplest practical language – that of *first-order* arithmetic expressions, like $x + y$ and $a + (5 \cdot b) + \sin(3)$.

Rather than complicate our language by introducing binary operators (like $x + y$), unary operators (like $-x$), and n -ary function applications (like $f(x, y, z)$), we can instead normalize everything to *S-expressions* – or *sexprs* – of the form $(f\ e_1 \dots e_n)$. We can then uniformly write:

- binary operations like $x + y$ as binary sexprs $(+ x\ y)$,
- unary operations $-x$ as unary sexprs $(- x)$,
- function applications $f(x, y, z)$ as n -ary sexprs $(f\ x\ y\ z)$.

In particular, the *operator* f always comes first, and there is no order-of-operations – all bracketing is explicit. Restricting ourselves to *first-order* sexprs means that we always require operators f to be atomic symbols drawn from a fixed set O – in particular, we don’t support *partial application*, like $((\text{add } 2)\ 3)$, since that would require $f = (\text{add } 2)$ to be a compound expression.

Formally, we define our untyped *syntax* as follows:

Definition 1.1 (First-order S-expressions): Fixing

- a set of *variables* $x \in N$
- a set of *operations* $f \in O$
- a set of *constants* $c \in V$

we define the set of *first-order S-expressions* $e \in \text{STm}_N(O, V)$ to be given by the grammar

$$e ::= x \mid c \mid (f e^*)$$

We can define a typing relation over this syntax, parametrized by a set of types $A \in \mathcal{T}$, in the usual manner. We begin by fixing some conventions:

Definition 1.2 (Typing Context): Given a set of variables N and a set of types $\mathcal{T}$, we define the set of *contexts* $\Gamma : \text{Ctx}_N(\mathcal{T})$ to be the set of lists of bindings $x : A$, given by the grammar

$$\Gamma ::= \cdot \mid \Gamma, x : A \text{ where } x \notin \text{def}(\Gamma)$$

Here, $\text{def}(\Gamma)$ denotes the set of *defined variables* in Γ , defined as

$$\text{def}(\cdot) = \emptyset \qquad \text{def}(\Gamma, x : A) = \text{def}(\Gamma) \cup \{x\}$$

Given a context Γ , we can hence define a partial function from variables N to types $\mathcal{T}$ as follows:

$$(\text{pty}(\Gamma), x : A)(x) := A \qquad \text{pty}(\Gamma, y : A)(x) := \text{pty}(\Gamma)(x) \text{ where } x \in \text{def}(\Gamma)$$

It follows that we can view a context Γ as isomorphic to:

- a *finitely supported partial function* $\text{pty}(\Gamma) : N \rightarrow \mathcal{T}$ – where the support of Γ is the set of defined variables $\text{def}(\Gamma)$
- a *total order* on the set of defined variables $\text{def}(\Gamma)$ – given by the order in which they appear in the context

In general, we'll define $\Gamma(x) := \text{pty}(\Gamma)(x)$.

Definition 1.3 (Typed Constants): We define a set of *typed constants* $c \in V$ to be a set V equipped with a *typing relation* $V \rightarrow \mathcal{T}$ – written $c : A$ to mean “ c has type A ”.

Note that constants are typed with a *relation* – that is, the same constant c can be assigned multiple types A, B, C .

Definition 1.4 (Typed Operations): We define a set of *typed operations* $f \in O$ to be a set O equipped with a *typing relation* $O \times \mathcal{T}^* \rightarrow \mathcal{T}$ – written $f : [A_1, \dots, A_m] \rightarrow B$ to mean “ f takes inputs $[A_1, \dots, A_m]$ to output B ”.

We've now got what we need to define a well-typed first-order S-expressions:

Definition 1.5 (Typed First-order S-expressions): Fixing

- a set of *variables* $x \in N$

- a set of *typed operations* $f \in O$ equipped with a *typing relation* $O \times \mathcal{T}^* \rightarrow \mathcal{T}$ – where “ f takes inputs $[A_1, \dots, A_m]$ to output B ” is written $f : [A_1, \dots, A_m] \rightarrow B$
- a set of *typed constants* $c \in V$ equipped with a *typing relation* $V \rightarrow \mathcal{T}$ – where “ c has type A ” is written $c : A$

we define a typing relation on first-order S-expressions e via the following rules:

- **TODO 2: Var**
- **TODO 3: Const**
- **TODO 4: Op**

TODO 5: Subtyping, weakening and substitution

TODO 6: Define the denotational semantics

- **Pure**
- **Monadic**
- **Categorical**

TODO 7:

Writing plan:

- **Write out basic arithmetic definitions**
- **Write out effectful definitions**
- **Write out iter-definitions...**
- **Or should we *start* with λ -iter... just following the paper...**